

BOOTSTRAP 5 EXERCISES SOLUTION CODES

Exercise 1.1: Basic HTML Page Linked to Bootstrap 5 via CDN

This exercise sets up a standard HTML5 starter template and hooks it up to Bootstrap 5 using Content Delivery Networks (CDNs). The CSS link goes in the `<head>` so the styles load immediately, and the JavaScript bundle goes right before the closing `</body>` tag so it doesn't block the page from rendering.

The Code

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Bootstrap Page</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

  <div class="container mt-5">
    <h1 class="text-primary">Hello, Bootstrap 5!</h1>
    <p>This page is linked to Bootstrap via CDN — no downloads or local installations needed.</p>
    <button class="btn btn-success">It works!</button>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Breakdown of What's Happening:

- `meta name="viewport"`: Crucial for responsive web design. It tells mobile browsers to scale the page properly rather than rendering it like a giant desktop site.
- `container`: A core Bootstrap class that centers your content and adds responsive padding on the sides.
- `mt-5`: A utility class that adds a hefty margin to the top (`margin-top: 3rem`) so your content isn't smashed against the top of the screen.
- `text-primary`: Changes the text color to Bootstrap's standard blue.

- `btn btn-success`: Transforms a standard, ugly browser button into a clean, modern green button.

Exercise 1.2: Set up a Project Using npm or Downloaded Bootstrap Files

When you want to host Bootstrap files locally rather than relying on an external server, you generally choose between two workflows depending on your project type.

Option A: The npm Route (For Modern Build Workflows)

Use this option if you are building a modern web application using a bundler like **Vite**, **Webpack**, or **Parcel**.

1. Initialize and Install

Run these commands in your project's root terminal folder to create a `package.json` file and pull down Bootstrap along with its dependency, Popper.js:

```
Bash
npm init -y
npm install bootstrap @popperjs/core
```

2. Import into Your JavaScript Entry Point

Inside your main JavaScript file (often `main.js` or `index.js`), import the compiled CSS and JS files directly from the `node_modules` folder:

```
JavaScript
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

Your bundler will automatically see these imports and compile them into your final production build.

Option B: The Downloaded Files Route (For Traditional, No-Build Projects)

Use this option if you want a simple, static website without installing Node.js or messing with terminal commands. It keeps your site fully functional even if you are working offline.

1. Download the Files

Go to getbootstrap.com, navigate to the **Download** section, and download the **"Compiled CSS and JS"** ZIP file.

2. Arrange Your Folder Structure

Unzip the folder and place it directly next to your `index.html` file. Your project folder should look something like this:

```
Plaintext
my-project/
├── bootstrap-5.3.3-dist/
│   ├── css/
│   │   └── bootstrap.min.css
│   ├── js/
│   │   └── bootstrap.bundle.min.js
└── index.html
```

3. Update Your HTML Links

Instead of pointing to a web URL, point your HTML tags to the local paths inside your project folder:

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Local Bootstrap Page</title>

  <link href="bootstrap-5.3.3-dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

  <div class="container mt-5">
    <h1 class="text-primary">Hello, Local Bootstrap!</h1>
    <p>This page loads Bootstrap entirely from your hard drive.</p>
    <button class="btn btn-success">Working Offline</button>
  </div>

  <script src="bootstrap-5.3.3-dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Exercise 2.1: Bootstrap directory structure

Plaintext

bootstrap-5.3.3-dist/

- | — css/
 - | | — bootstrap.css # unminified, human-readable
 - | | — bootstrap.min.css # minified — use in production
 - | | — bootstrap-grid.css # grid system only
 - | | — bootstrap-utilities.css # utility classes only
 - | | — bootstrap-reboot.css # CSS reset only
- | — js/
 - | | — bootstrap.js # plain JS, needs Popper loaded separately
 - | | — bootstrap.min.js
 - | | — bootstrap.bundle.js # JS + Popper bundled
 - | | — bootstrap.bundle.min.js # use this — bundle + minified
- | — icons/ # only if downloaded separately
 - | — font/
 - | — bootstrap-icons.svg

css/ — Contains layout, components, and utilities. Use the split files (`bootstrap-grid.css`, `bootstrap-utilities.css`) if you only need part of Bootstrap and want a smaller download size.

js/ — Powers interactive components like modals, dropdowns, tooltips, and accordions. Always prefer the bundle version because it includes Popper.js (which dropdowns and tooltips need for positioning), saving you from loading it as a separate script.

icons/ — Holds icon font/SVG files. This is only present if you downloaded Bootstrap Icons as its own package rather than pulling them from a CDN.

Rule of thumb: > * **Development/Debugging:** Use the unminified `bootstrap.css`.

Production: Use `bootstrap.min.css` + `bootstrap.bundle.min.js`.

Exercise 2.2: Include Bootstrap's JS plugins via `bootstrap.bundle.min.js`

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```

<head>

  <meta charset="UTF-8">

  <link href="bootstrap-5.3.3-dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body>

  <div class="dropdown m-4">

    <button class="btn btn-primary dropdown-toggle" data-bs-toggle="dropdown">

      Menu

    </button>

    <ul class="dropdown-menu">

      <li><a class="dropdown-item" href="#">Action</a></li>

      <li><a class="dropdown-item" href="#">Another action</a></li>

    </ul>

  </div>

  <script src="bootstrap-5.3.3-dist/js/bootstrap.bundle.min.js"></script>

</body>

</html>

```

Key point: Use `bootstrap.bundle.min.js`, not `bootstrap.min.js`. The bundle already includes Popper.js, so components like dropdowns and tooltips work with zero extra setup. You only need the `data-bs-toggle` attribute—no custom JavaScript required

Exercise 3.1: Responsive columns — 1 mobile / 2 tablet / 3 desktop

```

html
<div class="container">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-4">
      <div class="bg-primary text-white p-3 rounded text-center">Column A</div>
    </div>

```

```

<div class="col-12 col-md-6 col-lg-4">
  <div class="bg-primary text-white p-3 rounded text-center">Column B</div>
</div>
<div class="col-12 col-md-6 col-lg-4">
  <div class="bg-primary text-white p-3 rounded text-center">Column C</div>
</div>
</div>
</div>

```

How it works: `col-12` = full width by default (mobile), `col-md-6` = 2-per-row from the tablet breakpoint up, `col-lg-4` = 3-per-row from the desktop breakpoint up. `g-3` just adds gutter spacing between columns.

Exercise 3.2: Using `.container`, `.row`, `.col-*`

```

html
<div class="container">
  <div class="row">
    <div class="col bg-info text-white p-2 border">col</div>
    <div class="col bg-info text-white p-2 border">col</div>
    <div class="col bg-info text-white p-2 border">col</div>
  </div>
</div>

```

- `.container` — caps max width and centers content on the page, with responsive side padding.
- `.row` — creates a flex row and applies negative margins so column padding lines up correctly.
- `.col` (no number) — Bootstrap splits the remaining space evenly among however many `.col` elements are in the row.

Exercise 4.1: Sidebar (`col-md-3`) + content (`col-md-9`)

```

<div class="container">
  <div class="row">
    <div class="col-md-3 bg-secondary text-white p-3">Sidebar</div>
    <div class="col-md-9 bg-light p-3 border">Main content area</div>
  </div>
</div>

```

Sidebar takes 3 of 12 columns, content takes the remaining 9 — this only kicks in at the md breakpoint, so on mobile both stack full-width.

Exercise 4.2: Four equal columns (`col-sm-3`)

```

<div class="container">
  <div class="row g-2">
    <div class="col-sm-3"><div class="bg-warning p-3 text-center">1</div></div>
    <div class="col-sm-3"><div class="bg-warning p-3 text-center">2</div></div>
    <div class="col-sm-3"><div class="bg-warning p-3 text-center">3</div></div>
    <div class="col-sm-3"><div class="bg-warning p-3 text-center">4</div></div>
  </div>
</div>

```

```
</div>
</div>
```

Four `col-sm-3` blocks add up to 12, so they sit evenly in one row from the small breakpoint up, and stack on very narrow phones.

Exercise 5.1: justify-content-center + align-items-center

```
<div class="container">
  <div class="row justify-content-center align-items-center" style="min-height:120px;">
    <div class="col-4">
      <div class="bg-success text-white p-3 text-center rounded">Centered</div>
    </div>
  </div>
</div>
```

`justify-content-center` centers the column horizontally, `align-items-center` centers it vertically within the row's height — you need a `min-height` on the row to actually see the vertical centering.

Exercise 5.2: Reordering with order-md-*

```
<div class="container">
  <div class="row">
    <div class="col-md-6 order-md-2 bg-danger text-white p-3">First in HTML, shows 2nd on
desktop</div>
    <div class="col-md-6 order-md-1 bg-dark text-white p-3">Second in HTML, shows 1st on desktop</div>
  </div>
</div>
```

On mobile they display in normal source order. From md up, `order-md-1` and `order-md-2` swap their visual position without touching the HTML.

Exercise 6/7 (Flexbox) 7.1: Navbar using d-flex, flex-column, flex-md-row

```
<nav class="d-flex flex-column flex-md-row justify-content-between align-items-center bg-primary
text-white p-3 rounded gap-2">
  <span class="fw-bold">MyBrand</span>
  <div class="d-flex flex-column flex-md-row gap-3">
    <a class="text-white text-decoration-none" href="#">Home</a>
    <a class="text-white text-decoration-none" href="#">About</a>
    <a class="text-white text-decoration-none" href="#">Contact</a>
  </div>
</nav>
```

`flex-column` stacks everything vertically by default (mobile-first), then `flex-md-row` switches to a horizontal row from the md breakpoint up.

Exercise 7.2 (Flexbox): justify-content-between + align-items-center in a card

```
<div class="card">
  <div class="card-body d-flex justify-content-between align-items-center">
    <span>Order #1042</span>
    <span class="badge bg-success">Paid</span>
  </div>
</div>
```

`justify-content-between` pushes the two children to opposite ends, `align-items-center` keeps them vertically aligned even if they're different heights.

Exercise 7.1 (Typography): display, lead, text-muted, fw-bold

```
<h1 class="display-1">Display 1</h1>
<p class="lead">This is a lead paragraph, larger and lighter than normal body text.</p>
<p class="text-muted">This text is muted for secondary information.</p>
<p class="fw-bold">This text is bold.</p>
```

`display-1` through `display-6` are oversized, lightweight headings meant for hero sections. `lead` bumps up font size and weight slightly for intro paragraphs. `text-muted` grays text out for secondary info. `fw-bold` just sets font-weight to bold.

Exercise 7.2 (Typography): text-uppercase / lowercase / capitalize

```
<p class="text-uppercase">this text is uppercase</p>
<p class="text-lowercase">THIS TEXT IS LOWERCASE</p>
<p class="text-capitalize">this text is capitalized</p>
```

These are purely CSS `text-transform` utilities — they don't change the actual text content, just how it's displayed.

Exercise 8.1: Registration form

```
<form>
  <div class="mb-3">
    <label class="form-label" for="regName">Full name</label>
    <input type="text" class="form-control" id="regName" placeholder="Jane Doe">
  </div>

  <div class="mb-3">
    <label class="form-label" for="regPassword">Password</label>
    <div class="input-group">
      <span class="input-group-text">🔒 </span>
      <input type="password" class="form-control" id="regPassword" placeholder="Password">
    </div>
  </div>
```



```

<div class="form-check mb-3">
  <input class="form-check-input" type="checkbox" id="agreeTerms">
  <label class="form-check-label" for="agreeTerms">I agree to the terms and conditions</label>
</div>

```

```

<button type="submit" class="btn btn-primary">Register</button>
</form>

```

form-control styles standard inputs, form-check handles checkboxes/radios with proper spacing, and input-group lets you prefix/suffix an input with an icon or text (like the lock symbol here).

Exercise 8.2: Login form using form-floating

```

<form>
  <div class="form-floating mb-3">
    <input type="email" class="form-control" id="floatEmail" placeholder="name@example.com">
    <label for="floatEmail">Email address</label>
  </div>

  <div class="form-floating mb-3">
    <input type="password" class="form-control" id="floatPassword" placeholder="Password">
    <label for="floatPassword">Password</label>
  </div>

  <button type="submit" class="btn btn-outline-primary">Log in</button>
</form>

```

With form-floating, the label sits inside the input as placeholder-style text, then floats up above it once the field has focus or content. The input has to come before the label in the markup for this to work.

Exercise 9.1: All contextual button classes

```

<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-light">Light</button>
<button class="btn btn-dark">Dark</button>
<button class="btn btn-outline-primary">Outline Primary</button>
<button class="btn btn-outline-danger">Outline Danger</button>

```

Solid variants (btn-primary etc.) are filled buttons; btn-outline-* variants are transparent with a colored border and text, filling in with color on hover.

Exercise 9.2: Button groups + toggle buttons

```

<div class="btn-group" role="group">

```

```

<button class="btn btn-outline-secondary">Left</button>
<button class="btn btn-outline-secondary">Middle</button>
<button class="btn btn-outline-secondary">Right</button>
</div>

<div class="btn-group" role="group">
  <input type="checkbox" class="btn-check" id="toggle1" autocomplete="off">
  <label class="btn btn-outline-success" for="toggle1">Toggle 1</label>

  <input type="checkbox" class="btn-check" id="toggle2" autocomplete="off">
  <label class="btn btn-outline-success" for="toggle2">Toggle 2</label>
</div>

```

btn-group visually merges adjacent buttons (shared borders, no gaps). For toggle buttons, btn-check hides an actual checkbox/radio input and turns its <label> into the clickable, style-able button.

Exercise 10.1: Responsive navbar with logo, links, search

```

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Logo</a>

    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMain">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="navMain">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Services</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>

      <form class="d-flex" role="search">
        <input class="form-control me-2" type="search" placeholder="Search">
        <button class="btn btn-outline-light" type="submit">Search</button>
      </form>
    </div>
  </div>
</nav>

```

navbar-expand-lg means the navbar stays expanded (full menu visible) from the lg breakpoint up, and collapses into the hamburger toggle below that. This needs bootstrap.bundle.min.js loaded for the toggle button to actually work.

Exercise 10.2: nav-tabs / nav-pills

```

<ul class="nav nav-tabs mb-3">
  <li class="nav-item"><a class="nav-link active" href="#">Tab One</a></li>

```

```

<li class="nav-item"><a class="nav-link" href="#">Tab Two</a></li>
<li class="nav-item"><a class="nav-link" href="#">Tab Three</a></li>
</ul>

```

```

<ul class="nav nav-pills">
  <li class="nav-item"><a class="nav-link active" href="#">Pill One</a></li>
  <li class="nav-item"><a class="nav-link" href="#">Pill Two</a></li>
  <li class="nav-item"><a class="nav-link" href="#">Pill Three</a></li>
</ul>

```

nav-tabs gives a classic bordered-tab look; nav-pills gives rounded, pill-shaped buttons instead. Both just need the active class on whichever link is currently selected — actual tab-content switching would need data-bs-toggle="tab" plus matching tab-pane content divs if you want full tabbed panels.

Exercise 11.1: Profile card

```

<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Jane Doe</h5>
    <p class="card-text">Frontend developer who loves building accessible, responsive UIs.</p>
    <a href="#" class="btn btn-primary">View Profile</a>
  </div>
</div>

```

card-img-top locks the image to the top edge with matching rounded corners; card-body holds the padded content underneath.

Exercise 11.2: Media object layout (image left, content right)

```

<div class="d-flex">
  
</div>
<div>
  <h6 class="mt-0 mb-1">Media heading</h6>
  <p class="mb-0">Body text sits to the right of the image using a flex row.</p>
</div>
</div>

```

Bootstrap 4's .media class was dropped in Bootstrap 5 — this d-flex + me-3 pattern is the current recommended replacement.

Exercise 12.1: Margin/padding utilities

```

<div class="bg-info p-2">Base box with p-2</div>
<div class="bg-info m-3">Box with m-3</div>
<div class="bg-info mt-4">Box with mt-4 (margin-top only)</div>
<div class="bg-info py-5">Box with py-5 (padding top+bottom)</div>

```

Format is {property}{sides}-{size}. m=margin, p=padding. Sides: t(top), b(bottom), s(start/left), e(end/right), x(left+right), y(top+bottom), blank(all sides). Sizes go 0–5, plus auto.

Exercise 12.2: Pricing section

```
<div class="row text-center g-4">
  <div class="col-md-4">
    <div class="border rounded p-4">
      <h5 class="mb-3">Basic</h5>
      <h2 class="mb-3">$9<small class="text-muted fs-6">/mo</small></h2>
      <button class="btn btn-outline-primary mt-2">Choose</button>
    </div>
  </div>

  <div class="col-md-4">
    <div class="border rounded p-4 bg-primary text-white">
      <h5 class="mb-3">Pro</h5>
      <h2 class="mb-3">$29<small class="fs-6">/mo</small></h2>
      <button class="btn btn-light mt-2">Choose</button>
    </div>
  </div>

  <div class="col-md-4">
    <div class="border rounded p-4">
      <h5 class="mb-3">Enterprise</h5>
      <h2 class="mb-3">$99<small class="text-muted fs-6">/mo</small></h2>
      <button class="btn btn-outline-primary mt-2">Choose</button>
    </div>
  </div>
</div>
```

g-4 gives even gutter spacing between the three cards, and consistent mb-3 / mt-2 inside each card keeps the internal spacing uniform.

Exercise 13.1: Dashboard using bg-* and text colors

```
<div class="row g-2 text-center">
  <div class="col-md-3"><div class="bg-primary text-white p-3 rounded">Primary</div></div>
  <div class="col-md-3"><div class="bg-warning text-dark p-3 rounded">Warning</div></div>
  <div class="col-md-3"><div class="bg-success text-white p-3 rounded">Success</div></div>
  <div class="col-md-3"><div class="bg-danger text-white p-3 rounded">Danger</div></div>
</div>
```

Note bg-warning and bg-light use text-dark, not text-white — they're light backgrounds, so dark text keeps them readable.

Exercise 13.2: bg-gradient + bg-dark + white text

```
<div class="bg-dark bg-gradient text-white p-4 rounded text-center">
```

This box uses a subtle gradient overlay on top of the dark background.

```
</div>
```

`bg-gradient` just adds a subtle linear-gradient overlay on top of whatever `bg-*` color is already applied — it's a modifier, not a standalone class.

Exercise 14.1: `d-none`, `d-md-block`, `d-lg-flex`

```
<div class="alert alert-secondary d-none d-md-block">  
  Visible from the md breakpoint upward (tablet+).  
</div>
```

```
<div class="alert alert-info d-none d-lg-flex justify-content-between">  
  <span>Visible from the lg breakpoint upward (desktop+), laid out as flex.</span>  
  <span>→</span>  
</div>
```

`d-none` hides by default; the breakpoint-specific classes (`d-md-block`, `d-lg-flex`) override that hiding once the screen reaches that width — this is the standard "hide on mobile, show on desktop" pattern.

Exercise 14.2: Sidebar visible tablet and up

```
<div class="row">  
  <div class="col-md-3 d-none d-md-block bg-secondary text-white p-3">Sidebar</div>  
  <div class="col-md-9 bg-light p-3 border">Main content</div>  
</div>
```

The sidebar column is hidden by default and only appears from md up — on mobile, the content area alone takes the full width.

Exercise 15.1: border utilities + rounded-circle

```
  
<div class="border p-2">border</div>  
<div class="border border-primary p-2">border-primary</div>  
<div class="border border-3 p-2">border-3</div>
```

`border` adds a default 1px gray border, `border-{color}` recolors it, `border-{1 to 5}` controls thickness, and `rounded-circle` makes a square image perfectly circular.

Exercise 15.2: shadow, shadow-lg, rounded-pill

```
<div class="card shadow p-3">shadow</div>  
<div class="card shadow-lg p-3">shadow-lg</div>  
<button class="btn btn-primary rounded-pill">Pill Button</button>
```

`shadow-sm`, `shadow`, and `shadow-lg` scale from subtle to pronounced drop shadows. `rounded-pill` maxes out the `border-radius` so the element ends in fully rounded ends.

Exercise 16.1: Fixed footer

```
<footer class="position-fixed bottom-0 start-0 w-100 bg-dark text-white text-center py-2">  
  © 2026 My Site  
</footer>
```

`position-fixed` pulls the element out of normal document flow and pins it relative to the viewport; `bottom-0 start-0 w-100` stretches it across the bottom edge of the screen so it stays visible while scrolling.

Exercise 16.2: position-relative / position-absolute badge overlay

```
<div class="position-relative d-inline-block">  
    
  <span class="position-absolute top-0 start-100 translate-middle badge rounded-pill bg-danger">  
    New  
  </span>  
</div>
```

The wrapper needs `position-relative` so the badge's `position-absolute` is calculated relative to it, not the whole page. `translate-middle` centers the badge exactly on the corner point instead of just tucking it inside.

Exercise 17.1: Social icons in a footer

```
<footer class="bg-dark text-white text-center p-3">  
  <a href="#" class="text-white mx-2"><i class="bi bi-facebook fs-4"></i></a>  
  <a href="#" class="text-white mx-2"><i class="bi bi-twitter-x fs-4"></i></a>  
  <a href="#" class="text-white mx-2"><i class="bi bi-instagram fs-4"></i></a>  
  <a href="#" class="text-white mx-2"><i class="bi bi-linkedin fs-4"></i></a>  
</footer>
```

Needs the Bootstrap Icons stylesheet linked in `<head>`:

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css">
```

Exercise 17.2: Icon-only buttons

```
<button class="btn btn-outline-primary"><i class="bi bi-search"></i></button>  
<button class="btn btn-outline-success"><i class="bi bi-check-lg"></i></button>  
<button class="btn btn-outline-danger"><i class="bi bi-trash"></i></button>  
<button class="btn btn-outline-secondary"><i class="bi bi-gear"></i></button>
```

Good practice: add `aria-label="Search"` (etc.) on icon-only buttons since there's no visible text for screen readers to announce.

Exercise 18.1: Modal popup

```
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#exampleModal">  
  Open Modal
```

```
</button>
```

```
<div class="modal fade" id="exampleModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Modal Title</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        Modal content goes here.
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
      </div>
    </div>
  </div>
</div>
```

Works entirely through `data-bs-toggle/data-bs-target` attributes — no custom JS, as long as `bootstrap.bundle.min.js` is loaded on the page.

Exercise 18.2: Collapsible accordion

```
<div class="accordion" id="accordionExample">
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button" type="button" data-bs-toggle="collapse"
data-bs-target="#collapseOne">
        Section One
      </button>
    </h2>
    <div id="collapseOne" class="accordion-collapse collapse show"
data-bs-parent="#accordionExample">
      <div class="accordion-body">Content for section one.</div>
    </div>
  </div>

  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" type="button" data-bs-toggle="collapse"
data-bs-target="#collapseTwo">
        Section Two
      </button>
    </h2>
    <div id="collapseTwo" class="accordion-collapse collapse" data-bs-parent="#accordionExample">
      <div class="accordion-body">Content for section two.</div>
    </div>
  </div>
</div>
```

</div>

`data-bs-parent` on each collapse panel is what makes the accordion close other sections automatically when a new one opens — remove it and multiple sections can stay open at once.

Exercise 19.1: Set up Bootstrap 5 with Sass via npm

```
mkdir bootstrap-sass-project && cd bootstrap-sass-project
npm init -y
npm install bootstrap sass --save-dev
```

Create `src/custom.scss`:

```
@import "../node_modules/bootstrap/scss/bootstrap";
```

Add a build script to `package.json`:

```
"scripts": {
  "build:css": "sass src/custom.scss dist/custom.css"
}
```

Exercise 19.2: Customize colors and border radius, then recompile

```
// src/custom.scss
```

```
// 1. Overrides FIRST
```

```
$primary: #7b2cbf;
$border-radius: 1rem;
$border-radius-lg: 1.5rem;
$border-radius-sm: 0.5rem;
```

```
// 2. THEN import Bootstrap so it picks up the overrides
```

```
@import "../node_modules/bootstrap/scss/bootstrap";
```

Recompile with:

```
npm run build:css
```

Order matters — variable overrides have to come before the `@import` line, or Bootstrap's own defaults win. Once compiled, every component referencing `$primary` (buttons, links, badges, form focus states) and every rounded corner picks up the new values automatically, which is the whole advantage of Sass customization over patching CSS after the fact.

GUNASHREE V

111623102066

RMKCET